

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Introducción a la programación 2



Manual Técnico Práctica 1

202330209 Milton Efrén Ajxup Ramos

Quetzaltenango, agosto 2026

Índice

Índice	2
Introducción	3
Diagrama de Clases	4
Diagrama Entidad Relación	5
Diagrama de Tablas	6
Manual Técnico	7
Mecánica para listas visuales	7
Bloqueo de Mesas Ocupadas	7
Restringir el Producto a la Orden	8
Pago Empleados	9

INTRODUCCIÓN

Objetivo

Otorgar al usuario el soporte y ayuda necesaria para que pueda hacer una correcta ejecución de la aplicación de java del estudiante Milton, que en esta ocasión es el archivo de la Práctica 1 para el laboratorio de Introducción a la Programación y Computación 2.

Requerimientos

Tener una versión de java instalada

Tener disponible el archivo .jar proporcionado por el estudiante. Esta es una aplicación de escritorio primero desde la terminal se debe estar en la carpeta que contiene el archivo del Proyecto, concretamente en la carpeta de /Practica1SS2026/target y desde ahí ejecutar el comando "java -jar Practica1SS2026-1.0-SNAPSHOT-jar-with-dependencies.jar", hacer esto ejecutará el programa estamos listos para comenzar.

Diagrama de Clases



Diagrama Entidad Relación

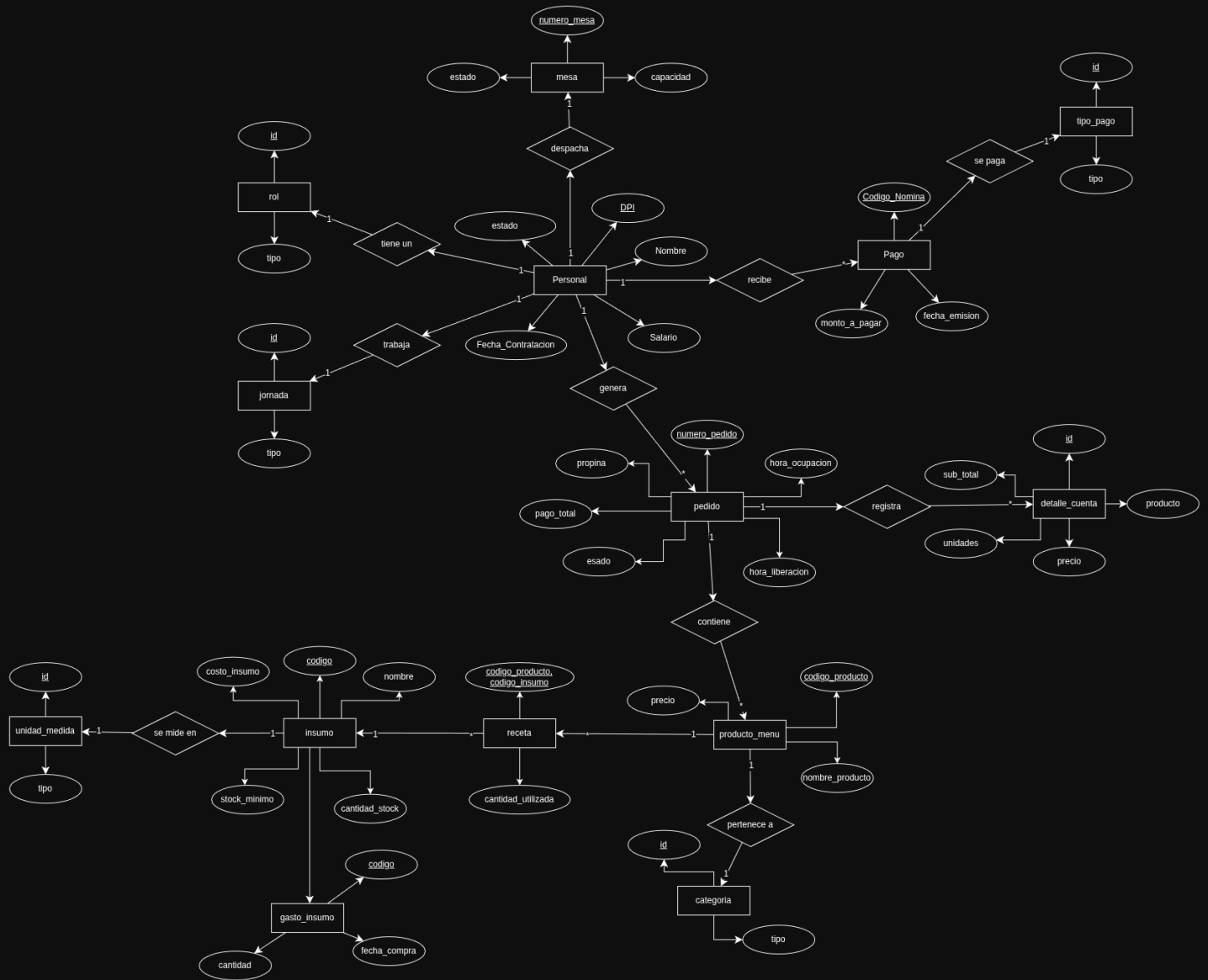
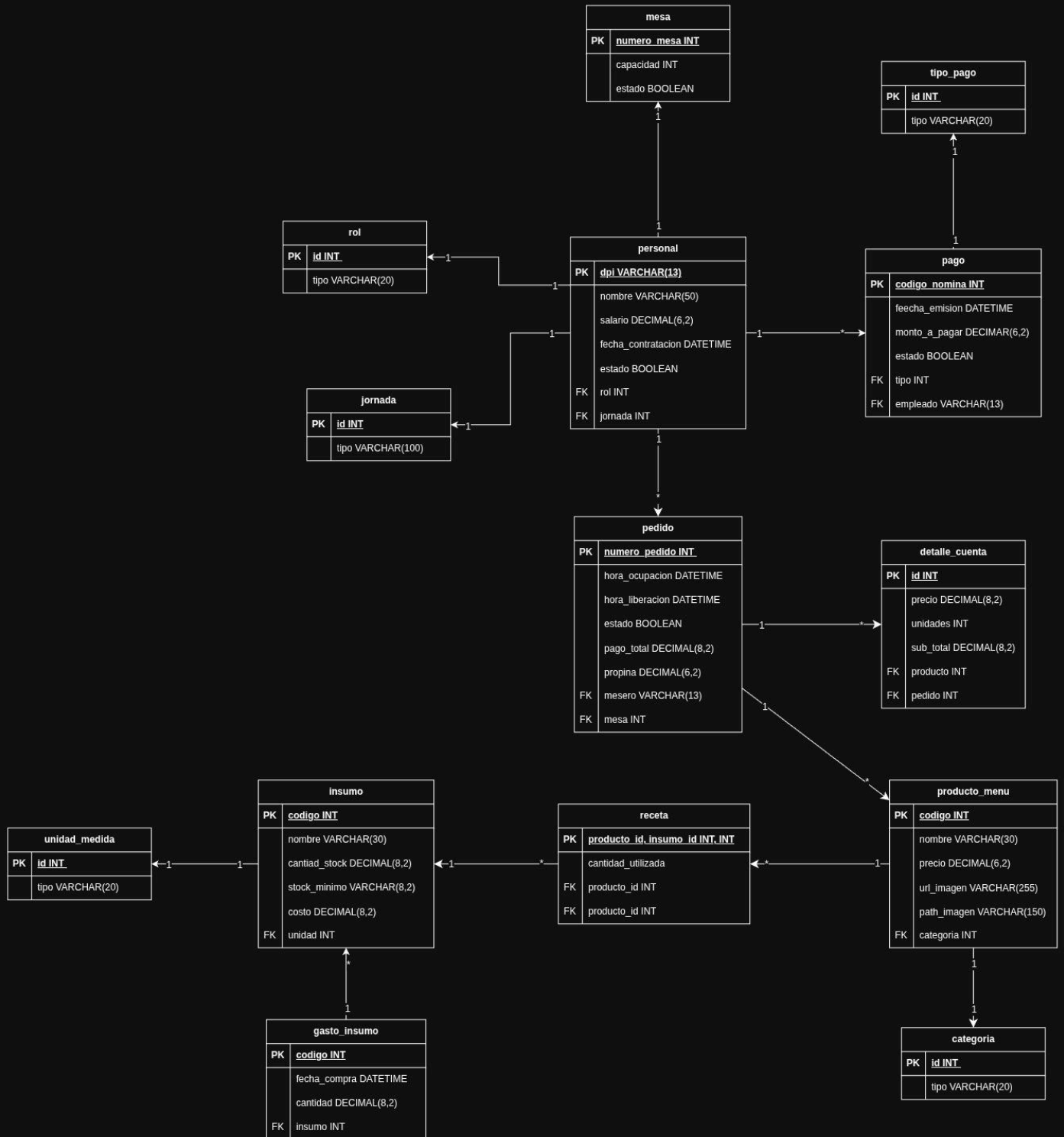


Diagrama de Tablas



Manual Técnico

Partes Importantes del Programa

Inicializador	Es clase que funciona como fábrica de los objetos principales que controlan las acciones al igual que la interfaz de usuario además de conectarlos
Controladores	Son los objetos que manejan las decisiones del usuario
Modelos	Son las clases que guardan los datos que provienen de la base de datos
Objetos que acceden a la base de datos DAOs	Son las clases encargadas de acceder a la base de dato mysql mediante consultas
Interfaz	Es la parte visual, creada para que el usuario se pueda comunicar con el programa

Mecánica para las listas visuales

En más de una ocasión en el programa se crean paneles con un formato definido, se crean múltiples para llenarlos con información de la base de datos, el JInternalFrame maneja un método que recibe estos paneles y los añade a otro que tiene un scrollpane, para evitar que la ventana se tuviera que expandir más de lo necesario además de agregarle un GridLayout con la cantidad de elementos que tiene que almacenar.

Un ejemplo

```
public void agregarMesa(PlantillaMesa plantillaMesa) {  
    jPanel1.add(plantillaMesa);  
}
```

Bloqueo de Mesas Ocupadas

Al momento de seleccionar un mesero, existen 2 posibilidades, que este mesero tenga un pedido activo, entonces solo podrá atender a la mesa en donde tiene ese pedido pendiente de cobrar y la otra posibilidad es que este libre, entonces se hace

otro chequeo que evita habilitar las mesa que estén ocupadas. Quitando la decisión de vista, así se ve el bloque.

```
public void elegirMesero(Personal mesero) throws AccesoALaDataException {
    ...cambia la vista
    Pedido pedido = meseroOcupado(mesero.getDpi());
    if (pedido != null) {
        controladorPago.setPedido(pedido);
    }
    for (int i = 0; i < mesas.size(); i++) {
        PlantillaMesa plantilla = plantillasMesa.get(i);
        if (pedido == null) {
            if (!plantilla.estaOcupado()) {
                plantilla.habilitarBoton(true);
            } else {
                plantilla.habilitarBoton(false);
            }
        }
    }
}
```

Restringir el Producto a la Orden

El programa guarda en memoria una lista de los Productos junto a los insumos del producto del pedido, aún sin actualizarlo en la base de datos, es hasta que se solicita el pedido que estos productos son restados en la base de datos.

Funciona así, cada vez que se selecciona un producto, el producto y los insumos que utiliza se guardan en listas, en cada adición se comprueba si la cantidad de insumos restante es suficiente para satisfacer la orden y si en algún momento el gasto de insumos llega a ser superior a la de los insumos disponibles, se denegará y evitará la adición del producto.

```
private boolean existenUnidades(int codigo, double cantidadUtilizada, double
cantidadRestante) {
    double suma;
    for (int i = 0; i < insumosPedido.size(); i++) {
        InsumoPedido actual = insumosPedido.get(i);
        if (actual.getCodigo() == codigo) {
            suma = actual.getCantidad() + cantidadUtilizada;
            cantidadRestante = cantidadRestante - suma;
            return cantidadRestante >= 0;
        }
    }
}
```



```

    }
}
return false;
}

```

Pago de Empleados

El pago de empleados funciona de tal forma que: primero evalúa la fecha y que sea una fecha válida, si es así, evalúa si es 10 o 25 del mes y genera las órdenes de pago de los empleados.

Dependiendo de si es 10 o 25, el sistema genera un registro calculando el porcentaje que tiene que pagar y registra la generación del pago en la base de datos.

El sistema omite los pagos de aquellos empleados que ya fueron generados, esto lo hace buscando en la base de datos y si existe un registro es porque ya se ha generado con anterioridad, entonces omite ese pago.

A la vez que verifica si tiene que generar pagos, busca pagos generados en esa fecha, si encuentra entonces los muestra y estos paneles, son los que presentan la opción de registrar que el pago se haya realizado.

```

    public void generarRegistrosPago(String diaTexto, String mesTexto, String
añoTexto) throws AccesoALaDataException, ErrorIngresarDatosException {
        procesarFecha(diaTexto, mesTexto, añoTexto);
        empleados = personaldao.getPersonal();
        servicioPagoEmpleado.limpiar();
        servicioPagoEmpleado.setFilas(empleados.size());
        String fecha = procesarFecha(diaTexto, mesTexto, añoTexto);
        if (dia == 10) {
            //Si es este pago es QUINCENA y en base de datos es 1
            generarPago(fecha, 1, 30);
        } else if (dia == 25) {
            //Si es este pago es FIN_DE_MES y en base de datos es 2
            generarPago(fecha, 2, 70);
        } else {
            servicioPagoEmpleado.mostrarError("No se genera nada, los pagos generan
los dias 10 y 25");
        }
    }
}

```

Uso de consultas sql

Para el manejo de los datos de la base de datos se utilizan consultas UPDATE o al crear nuevos registros se utilizan el INSERT, estas se utilizan con objeto PreparedStatement para enviar inyecciones sql, un ejemplo:

```
private final String actualizarMesa = "UPDATE mesa SET estado = ? WHERE  
numero_mesa = ?";
```

Y dentro de un método se llenan los campos, estos datos se recuperan casi en todo momento, de esta forma se muestra una versión actual de los distintos estados de los registros guardados.

Manejo de JOIN

Durante el proyecto existen varias ocasiones en las que se tiene que acceder a la información de distintas tablas, no es conveniente tener la información duplicada en las tablas, por ejemplo el nombre de los insumos que tiene un producto X, por esa razón existen las llaves foráneas, los JOIN utilizan esa relación que existe al agregar llaves foráneas y combina las tablas que se incluyen en la consulta, de esta forma se crean objetos con la información completa para mostrar al usuario sin necesidad de que las tablas tengan la información duplicada.

```
private final String INSUMOS_PRODUCTO = ""  
    SELECT pro.codigo AS codigo_producto,  
    pro.nombre AS nombre_producto,  
    ins.codigo AS codigo_insumo, ins.nombre AS nombre_insumo,  
    rec.cantidad_utilizada, uni.unidad  
    FROM producto_menu AS pro  
    JOIN receta AS rec ON pro.codigo = rec.producto_id  
    JOIN insumo AS ins ON rec.insumo_id = ins.codigo  
    JOIN unidad_medida AS uni ON ins.unidad = uni.id  
WHERE pro.codigo = ?"";
```